

ALUMA

Documentación de la Base de Datos

Modelo relacional - Versión 1.0

Objetivo: Documentar la estructura, relaciones y reglas principales de la base de datos del proyecto ALUMA para facilitar su mantenimiento y el desarrollo posterior de la API.

Fecha: 28 de septiembre de 2026

Sistema gestor: MariaDB / MySQL compatible

1. Introducción

La base de datos de ALUMA está diseñada para gestionar usuarios, autenticación, actividades, inscripciones, pagos y movimientos contables. El modelo separa los datos operativos de las tablas de catálogo, como estados, roles, métodos de pago y tipos de movimiento.

Esta primera versión prioriza un diseño sencillo, normalizado y suficientemente flexible para crecer sin introducir complejidad innecesaria. La aplicación utilizará la base de datos a través de una API, por lo que las reglas de negocio que no puedan expresarse únicamente mediante restricciones SQL se implementarán en la capa de servicio del backend.

Criterio general: Las tablas de catálogo contienen valores reutilizables y las tablas operativas almacenan las operaciones reales. Esto evita repetir textos y facilita los cambios futuros.

2. Índice de entidades

Entidad	Finalidad
Usuario	Datos personales y administrativos de cada usuario.
RolUsuario	Roles disponibles para autorización.
EstadoUsuario	Estado funcional de la cuenta y permisos generales.
TipoDocumentIdentificacion	Catálogo de tipos de documento.
CuentaAutenticacion	Credenciales locales y vínculos OAuth.
Actividad	Actividades o eventos organizados por ALUMA.
EstadoActividad	Estados posibles de una actividad.
InscripcionActividad	Relación entre usuario y actividad, con datos del pago.
EstadoPago	Estados del proceso de pago de una inscripción.
MetodoPago	Métodos utilizados para efectuar un pago.
MovimientoContable	Registro económico generado por inscripciones, devoluciones o ajustes.
TipoMovimiento	Clasificación de los movimientos contables.

3. Modelo relacional

El siguiente diagrama representa las relaciones entre las tablas de la versión 1.0. La nomenclatura del diagrama procede de la implementación real en MariaDB.

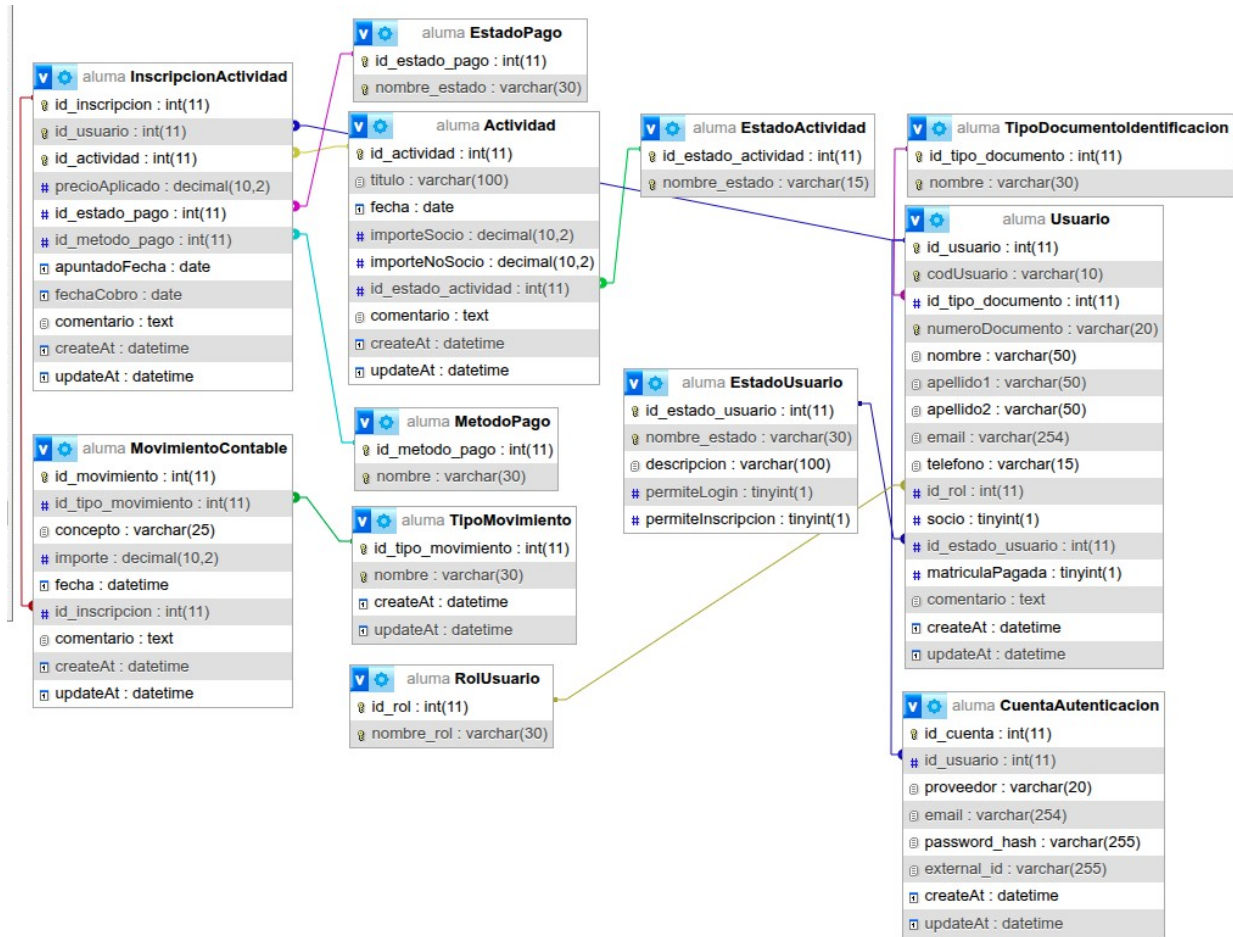


Figura 1. Modelo relacional de ALUMA.

4. Gestión de usuarios y autenticación

4.1 Usuario

Tabla central de personas registradas en ALUMA. Contiene datos identificativos, información de contacto, rol, estado y condición de socio.

Campo	Tipo	Descripción
id_usuario	INT	Clave primaria autoincremental.
codUsuario	VARCHAR(10)	Código interno del usuario.
id_tipo_documento	INT	FK al tipo de documento de identificación.
numeroDocumento	VARCHAR(20)	Número del documento de identificación.
nombre	VARCHAR(50)	Nombre del usuario.
apellido1	VARCHAR(50)	Primer apellido.
apellido2	VARCHAR(50)	Segundo apellido.
email	VARCHAR(254)	Correo electrónico principal.
telefono	VARCHAR(15)	Teléfono de contacto.
id_rol	INT	FK a RolUsuario.
socio	TINYINT(1)	Indica si el usuario tiene condición de socio.
id_estado_usuario	INT	FK a EstadoUsuario.
matriculaPagada	TINYINT(1)	Indica si la matrícula está pagada.
comentario	TEXT	Observaciones internas.
createAt	DATETIME	Fecha y hora de creación.
updateAt	DATETIME	Fecha y hora de última modificación.

Relaciones

- N usuarios pueden compartir un mismo RolUsuario.
- N usuarios pueden compartir un mismo EstadoUsuario.
- N usuarios pueden compartir un mismo TipoDocumentoIdentificacion.
- Un Usuario puede tener una o varias CuentaAutenticacion.
- Un Usuario puede tener varias InscripcionActividad.

Reglas principales

- La autorización se determina mediante el rol y el estado del usuario.
- La condición de socio se utiliza para decidir el precio aplicable de una actividad.
- Los datos de autenticación no se almacenan directamente en esta tabla.

4.2 RolUsuario

Catálogo de roles utilizados por la aplicación para controlar permisos y acceso a funcionalidades.

Campo	Tipo	Descripción
id_rol	INT	Clave primaria autoincremental.
nombre_rol	VARCHAR(30)	Nombre del rol, por ejemplo Usuario, Operador o Administrador.

Relaciones

- Un RolUsuario puede estar asociado a muchos Usuario.

4.3 EstadoUsuario

Catálogo de estados de usuario. Permite controlar el comportamiento de una cuenta sin eliminarla.

Campo	Tipo	Descripción
id_estado_usuario	INT	Clave primaria autoincremental.
nombre_estado	VARCHAR(30)	Nombre del estado.
descripcion	VARCHAR(100)	Descripción funcional.
permiteLogin	TINYINT(1)	Indica si el usuario puede iniciar sesión.
permiteInscripcion	TINYINT(1)	Indica si puede inscribirse en actividades.

Relaciones

- Un EstadoUsuario puede estar asignado a muchos Usuario.

4.4 TipoDocumentoIdentificacion

Catálogo de documentos válidos para identificar a un usuario.

Campo	Tipo	Descripción
id_tipo_documento	INT	Clave primaria autoincremental.
nombre	VARCHAR(30)	Nombre del tipo de documento, por ejemplo DNI, NIE o Pasaporte.

Relaciones

- Un TipoDocumentoIdentificacion puede estar asociado a muchos Usuario.

4.5 CuentaAutenticacion

Tabla separada para almacenar la forma de acceso de cada usuario. Permite combinar login local y proveedores OAuth sin duplicar usuarios.

Campo	Tipo	Descripción
id_cuenta	INT	Clave primaria autoincremental.
id_usuario	INT	FK a Usuario.
proveedor	VARCHAR(20)	Proveedor de autenticación: Local, Google, Microsoft, Facebook, etc.
email	VARCHAR(254)	Correo utilizado por la cuenta de autenticación.
password_hash	VARCHAR(255)	Hash de contraseña. Puede ser NULL en OAuth.

external_id	VARCHAR(255)	Identificador proporcionado por el proveedor OAuth. Puede ser NULL para autenticación local.
createAt	DATETIME	Fecha de creación.
updateAt	DATETIME	Última modificación.

Relaciones

- Un Usuario puede tener varias CuentaAutenticacion.
- Cada CuentaAutenticacion pertenece a un único Usuario.

Reglas principales

- Nunca se almacena la contraseña en texto plano; únicamente su hash.
- Para cuentas OAuth, external_id identifica al usuario dentro del proveedor.
- La tabla permite vincular distintos proveedores a un mismo Usuario.

5. Actividades e inscripciones

5.1 Actividad

Representa una actividad o evento organizado por ALUMA. Puede establecer precios distintos para socios y no socios.

Campo	Tipo	Descripción
id_actividad	INT	Clave primaria autoincremental.
titulo	VARCHAR(100)	Título de la actividad.
fecha	DATE	Fecha prevista. Puede ser NULL mientras no esté definida.
importeSocio	DECIMAL(10,2)	Precio para socios.
importeNoSocio	DECIMAL(10,2)	Precio para no socios.
id_estado_actividad	INT	FK a EstadoActividad.
comentario	TEXT	Observaciones.
createAt	DATETIME	Fecha de creación.
updateAt	DATETIME	Última modificación.

Relaciones

- Cada Actividad tiene un EstadoActividad.
- Una Actividad puede tener muchas InscripcionActividad.

Reglas principales

- El precio final cobrado se conserva en InscripcionActividad.precioAplicado para mantener el histórico aunque cambien después los precios de la actividad.

5.2 EstadoActividad

Catálogo de estados de una actividad.

Campo	Tipo	Descripción
id_estado_actividad	INT	Clave primaria autoincremental.
nombre_estado	VARCHAR(15)	Nombre del estado, por ejemplo Activo, Inactivo o Pendiente.

Relaciones

- Un EstadoActividad puede aplicarse a muchas Actividad.

5.3 InscripcionActividad

Tabla asociativa que registra la inscripción de un usuario a una actividad y el estado económico de esa inscripción.

Campo	Tipo	Descripción
id_inscripcion	INT	Clave primaria autoincremental.
id_usuario	INT	FK a Usuario.

id_actividad	INT	FK a Actividad.
precioAplicado	DECIMAL(10,2)	Precio fijado para esta inscripción.
id_estado_pago	INT	FK a EstadoPago.
id_metodo_pago	INT	FK a MetodoPago. Puede ser NULL mientras no exista pago.
apuntadoFecha	DATE	Fecha en la que el usuario se apuntó.
fechaPago	DATE	Fecha del pago. Puede ser NULL hasta que se produzca.
comentario	TEXT	Observaciones de la inscripción.
createAt	DATETIME	Fecha de creación.
updateAt	DATETIME	Última modificación.

Relaciones

- Cada inscripción pertenece a un Usuario y una Actividad.
- Cada inscripción tiene un EstadoPago.
- El MetodoPago puede no existir todavía si el pago está pendiente.
- Una inscripción puede originar varios MovimientoContable.

Reglas principales

- La combinación usuario + actividad debe impedir duplicar la misma inscripción.
- Al confirmarse un pago, la aplicación genera un movimiento contable de tipo Cobro.
- Una devolución genera un movimiento contable independiente, preservando el histórico.

5.4 EstadoPago

Catálogo de estados del pago de una inscripción.

Campo	Tipo	Descripción
id_estado_pago	INT	Clave primaria autoincremental.
nombre_estado	VARCHAR(30)	Estado del pago, por ejemplo Pendiente, Pagado, Devolución o Cancelado.

Relaciones

- Un EstadoPago puede utilizarse en muchas InscripcionActividad.

5.5 MetodoPago

Catálogo de métodos aceptados para efectuar pagos.

Campo	Tipo	Descripción
id_metodo_pago	INT	Clave primaria autoincremental.
nombre	VARCHAR(30)	Nombre del método, por ejemplo Bizum, Efectivo, Transferencia o Tarjeta.

Relaciones

- Un MetodoPago puede utilizarse en muchas InscripcionActividad.

6. Registro contable

6.1 MovimientoContable

Registra los movimientos económicos asociados a las operaciones de ALUMA. En esta versión está orientado principalmente a cobros, devoluciones y ajustes relacionados con inscripciones.

Campo	Tipo	Descripción
id_movimiento	INT	Clave primaria autoincremental.
id_tipo_movimiento	INT	FK a TipoMovimiento.
concepto	VARCHAR(25)	Descripción breve del movimiento.
importe	DECIMAL(10,2)	Importe del movimiento.
fecha	DATETIME	Fecha y hora del movimiento.
id_inscripcion	INT	FK a InscripcionActividad. Puede ser NULL para movimientos no vinculados a una inscripción concreta.
comentario	TEXT	Observaciones.
createAt	DATETIME	Fecha de creación.
updateAt	DATETIME	Última modificación.

Relaciones

- Cada MovimientoContable tiene un TipoMovimiento.
- Una InscripcionActividad puede estar relacionada con cero, uno o varios MovimientoContable.

Reglas principales

- Los cobros, devoluciones y ajustes se registran como movimientos independientes.
- El histórico no debe reescribirse para simular una devolución; debe añadirse un nuevo movimiento.
- El saldo puede calcularse mediante la suma de los importes registrados.

6.2 TipoMovimiento

Catálogo que clasifica los movimientos contables.

Campo	Tipo	Descripción
id_tipo_movimiento	INT	Clave primaria autoincremental.
nombre	VARCHAR(30)	Nombre del tipo de movimiento.
createAt	DATETIME	Fecha de creación.
updateAt	DATETIME	Última modificación.

Relaciones

- Un TipoMovimiento puede ser utilizado por muchos MovimientoContable.

Reglas principales

- Valores iniciales previstos: Cobro, Devolución y Ajuste.

7. Resumen de relaciones

Origen	Cardinalidad	Destino	Interpretación
RolUsuario	1:N	Usuario	Un rol puede asignarse a muchos usuarios.
EstadoUsuario	1:N	Usuario	Un estado puede compartirse entre muchos usuarios.
TipoDocumentoIdentificacion	1:N	Usuario	Un tipo de documento puede utilizarse por muchos usuarios.
Usuario	1:N	CuentaAutenticacion	Un usuario puede disponer de varias formas de autenticación.
EstadoActividad	1:N	Actividad	Un estado puede aplicarse a muchas actividades.
Usuario	1:N	InscripcionActividad	Un usuario puede realizar varias inscripciones.
Actividad	1:N	InscripcionActividad	Una actividad puede tener muchas inscripciones.
EstadoPago	1:N	InscripcionActividad	Un mismo estado puede aparecer en muchas inscripciones.
MetodoPago	1:N	InscripcionActividad	Un método puede utilizarse en muchas inscripciones.
TipoMovimiento	1:N	MovimientoContable	Un tipo clasifica muchos movimientos.
InscripcionActividad	1:N	MovimientoContable	Una inscripción puede generar varios movimientos contables.

8. Reglas de negocio principales

Regla	Descripción
Identidad del usuario	Usuario concentra los datos personales. Las credenciales se separan en CuentaAutenticacion.
Autenticación múltiple	Un mismo usuario puede acceder mediante cuenta local u OAuth sin crear registros Usuario duplicados.
Control de acceso	RolUsuario define capacidades de autorización y EstadoUsuario permite bloquear login o inscripción sin borrar al usuario.
Precio histórico	precioAplicado conserva el precio real asignado al inscribirse, independientemente de cambios posteriores en Actividad.
Pago pendiente	id_metodo_pago y fechaPago pueden permanecer vacíos hasta que el pago se produzca.
Registro de cobro	Al pasar una inscripción a Pagado, el backend debe crear el MovimientoContable correspondiente.
Devoluciones	Una devolución debe generar un nuevo movimiento y no borrar el cobro original.
Trazabilidad	createAt y updateAt permiten conocer la creación y última modificación de los registros principales.
No duplicación de inscripción	Un usuario no debe poder tener dos inscripciones para la misma actividad.
Integridad referencial	Las claves foráneas garantizan que los registros relacionados existan antes de ser referenciados.

9. Flujos de ejemplo

9.1 Alta e inicio de sesión

1. Se crea el registro Usuario con sus datos personales y administrativos.
2. Se crea una CuentaAutenticacion vinculada al usuario.
3. Si el proveedor es Local, se almacena password_hash. Si es OAuth, se almacena external_id.
4. Durante el login se comprueba EstadoUsuario.permiteLogin y posteriormente el rol del usuario.

9.2 Inscripción y pago de una actividad

5. El usuario selecciona una Actividad disponible.
6. La aplicación decide el precio según la condición de socio y lo guarda en precioAplicado.
7. Se crea InscripcionActividad inicialmente con el estado de pago correspondiente.
8. Cuando se confirma el pago, se completan id_metodo_pago y fechaPago.
9. El backend registra un MovimientoContable de tipo Cobro asociado a la inscripción.

9.3 Devolución

10. La inscripción cambia al estado de pago definido para la devolución.
11. Se conserva el movimiento de cobro original.
12. Se crea un nuevo MovimientoContable de tipo Devolución con el importe correspondiente.
13. El historial permite reconstruir la operación completa sin perder información.

10. Consideraciones para el backend/API

Aspecto	Recomendación
Transacciones	Las operaciones que modifican inscripción y contabilidad al mismo tiempo deben ejecutarse dentro de una transacción SQL.
Validación	El backend debe validar campos obligatorios, formatos de email/documento y coherencia entre estado de pago, método y fecha.
Contraseñas	La contraseña nunca se guarda ni se registra en logs. Solo debe persistirse un hash robusto generado por una librería adecuada.
OAuth	El external_id debe tratarse como identificador del proveedor y utilizarse junto con proveedor para localizar la cuenta.
Eliminaciones	Para registros con histórico económico es preferible desactivar o conservar antes que eliminar físicamente.
Importes	DECIMAL(10,2) es apropiado para importes monetarios. No deben utilizarse FLOAT o DOUBLE para dinero.
Fechas	Se recomienda acordar una política única de zona horaria para los DATETIME de la API y documentarla en el backend.

11. Estado de la versión 1.0

La estructura definida cubre las necesidades previstas para la primera versión de ALUMA: gestión de usuarios, roles, estados, autenticación, actividades, inscripciones, pagos y registro económico básico. El modelo está preparado para que el siguiente paso del proyecto sea la implementación del backend/API en TypeScript.

Conclusión: La base de datos queda documentada como versión 1.0. Cualquier ampliación futura deberá realizarse mediante cambios controlados del esquema y actualización simultánea de esta documentación.

12. Glosario breve

Término	Significado
PK	Primary Key. Identificador único de una fila.
FK	Foreign Key. Campo que referencia la clave de otra tabla.
1:N	Relación uno a muchos.
NULL	El campo puede no contener valor.
NOT NULL	El campo requiere un valor.
UNIQUE	Restricción que impide valores duplicados.
AUTO_INCREMENT	Generación automática de identificadores numéricos.
OAuth	Mecanismo para iniciar sesión mediante un proveedor externo sin almacenar su contraseña.
Hash	Representación irreversible utilizada para almacenar contraseñas de forma segura.